WEDDINGOS

Database Design

MongoDB Atlas | Mongoose | Wedding-centric data model

Version 1.0

Prepared for the WeddingOS MVP

A practical database blueprint for managing wedding workspaces, events, guest families, RSVPs, expenses, vendors, tasks, galleries and access control.

DESIGN STATUS Recommended MVP baseline

Document overview

Item	Decision
Database	MongoDB Atlas
ODM	Mongoose
Architecture	Wedding-centric, multi-tenant data model
Core strategy	Separate collections for major entities; embed small bounded subdocuments
Relationship key	ObjectId references, with weddingId on wedding-specific records
Money representation	Integer paise (INR) rather than floating-point rupees
API architecture	REST API under /api/v1; modular monolith; controller-service-repository

1. Design goals

- Keep each wedding workspace logically isolated from every other wedding.
- Allow one global user account to participate in multiple weddings with different roles.
- Support family-level guest invitations and one RSVP per family per event.
- Preserve expense and payment history, while making paid and outstanding totals easy to calculate.
- Support event-based tasks and event-specific photo/video galleries.
- Keep the MVP understandable and maintainable without premature microservices or duplicated data.

2. Core modelling decisions

Decision	Chosen approach	Reason
Tenant boundary	Each wedding is an independent data boundary	Consistent isolation and simpler authorization.
Membership	Separate weddingMembers collection	Supports different roles across multiple weddings and immediate revocation.
Major entities	Separate collections	Avoids large unbounded wedding documents and supports direct querying.
Event invitations	Separate guestEventInvitations collection	Models the many-to-many family/event relationship.
RSVPs	Separate rsvps collection	One independently updateable response per family per event.
Categories	Separate categories collection	Reusable typed categories scoped to a wedding.
Payments	Separate payments collection	Preserves each instalment as an identifiable financial record.
Gallery files	Separate galleryFiles collection	File count can grow independently of

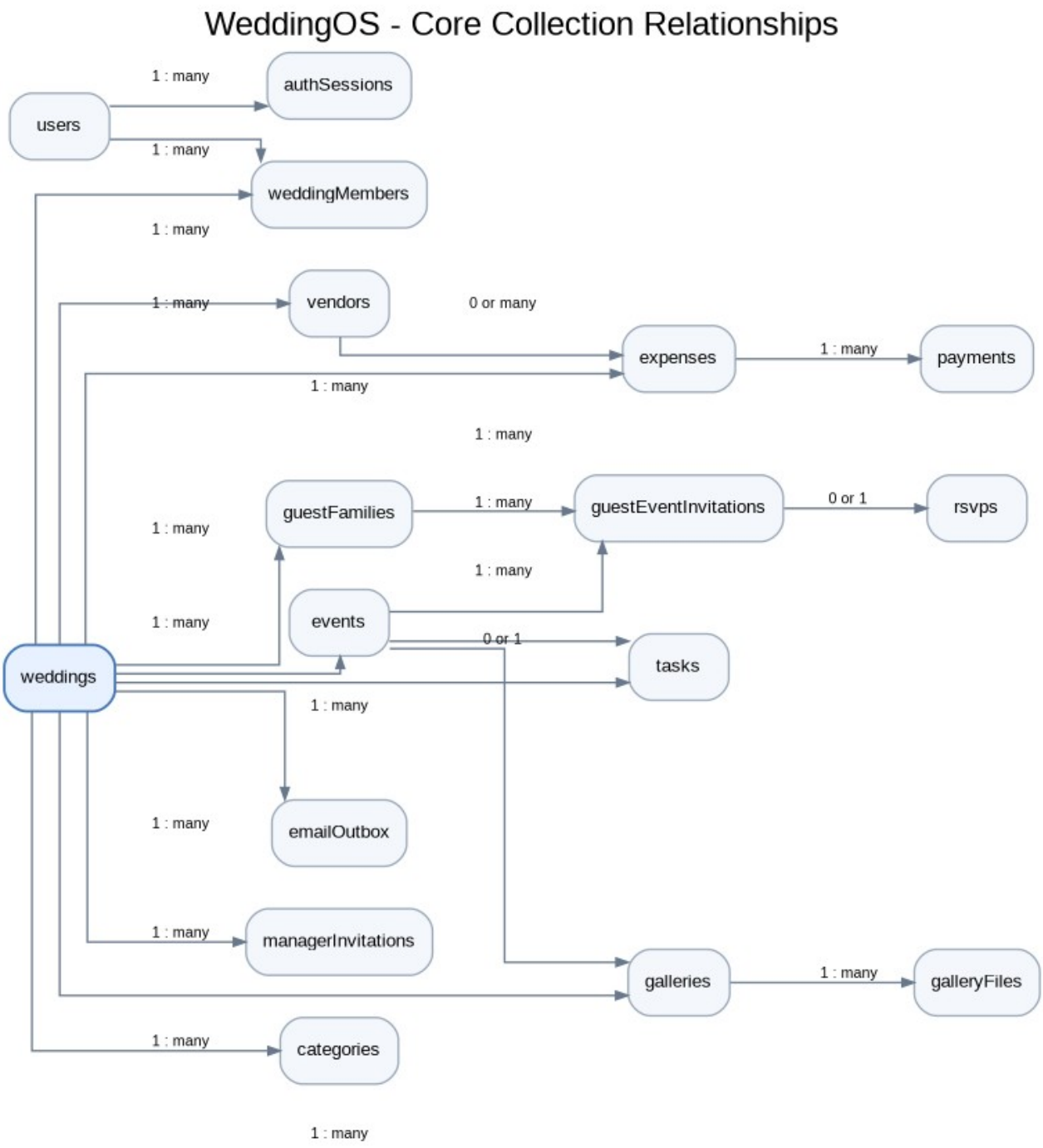
Decision	Chosen approach	Reason
		gallery configuration.

3. Collection inventory

Group	Collection	Responsibility
Accounts & access	users	Global user identity and login credentials.
Accounts & access	authSessions	Refresh-token session tracking and revocation.
Accounts & access	weddingMembers	Wedding-specific Admin/Manager membership.
Accounts & access	managerInvitations	Single-use, seven-day manager invitation links.
Wedding setup	weddings	Workspace identity, venue, lifecycle and public page.
Wedding setup	categories	Wedding-scoped guest, expense and vendor categories.
Events & guests	events	Event schedule and guest-facing/private information.
Events & guests	guestFamilies	One record per invited family.
Events & guests	guestEventInvitations	Family-to-event invitation relationships.
Events & guests	rsvps	Family response for a particular event.
Finance	expenses	Agreed expense records.
Finance	payments	Individual instalments against expenses.
Finance	vendors	Saved vendor contact and workflow information.
Operations	tasks	Event-specific tasks and assignees.
Media	galleries	Per-event gallery link configuration.
Media	galleryFiles	Cloudinary asset metadata for photos/videos.
System operations	emailOutbox	Durable guest email notification and retry records.

4. Relationship map

The diagram shows the principal relationships. A weddingId is also carried by each wedding-specific collection so the service layer can enforce tenant isolation.



Relationship summary

Parent	Child	Relationship
users	authSessions	One user can have many active or historical sessions.
users / weddings	weddingMembers	One membership links one user to one wedding.
weddings	events, guestFamilies, expenses, vendors, tasks, galleries	One wedding owns many workspace records.

Parent	Child	Relationship
guestFamilies / events	guestEventInvitations	Many-to-many relationship resolved by invitation documents.
guestEventInvitations	rsvps	An invitation may have zero or one RSVP.
expenses	payments	One expense can have many instalment records.
vendors	expenses	An expense may optionally reference a saved vendor.
events	tasks / galleries	An event can have many tasks and at most one gallery.

5. Detailed collection schemas

The examples are illustrative document shapes, not copy-paste-complete Mongoose schemas. ObjectId values, dates, enums, validation and indexes should be defined explicitly in the implementation.

5.1. users

Purpose: Global account identity. A user may own one wedding and manage other weddings.

Field	Description
emailNormalized	Lowercased/trimmed email; unique across users.
passwordHash	Optional for Google-only accounts; never store plaintext passwords.
googleId	Optional Google identity; unique when present.
emailVerifiedAt	Set after email verification or trusted Google authentication.
status	Account state, e.g. ACTIVE or DISABLED.

```
{
  _id: ObjectId,
  name: "Aditya Mishra",
  email: "aditya@example.com",
  emailNormalized: "aditya@example.com",
  passwordHash: "...",
  googleId: null,
  emailVerifiedAt: Date,
  status: "ACTIVE",
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Use a unique index on emailNormalized. Use a unique sparse index on googleId if accounts can have no Google identity.

5.2. authSessions

Purpose: Server-side refresh-token session records used for rotation and revocation.

Field	Description
userId	Reference to users._id.
refreshTokenHash	Hash of the refresh token; do not store the raw token.
expiresAt	Refresh session expiry; TTL cleanup can use this field.
revokedAt	Set when the session is revoked.
lastUsedAt	Last successful refresh/use timestamp.

```
{
  _id: ObjectId,
  userId: ObjectId,
  refreshTokenHash: "...",
  expiresAt: Date,
  revokedAt: null,
  lastUsedAt: Date,
  createdAt: Date
}
```

Design notes: Use refresh-token rotation. A TTL index on expiresAt is cleanup only; authorization must still check expiry and revocation.

5.3. weddingMembers

Purpose: Membership and role of a global user in a particular wedding.

Field	Description
weddingId + userId	References to the wedding and user; unique as a pair.
role	ADMIN or MANAGER.
status	ACTIVE or REMOVED.
removedAt / removedBy	Preserve membership removal history.
joinedAt	When membership became active.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  userId: ObjectId,
  role: "MANAGER",
  status: "ACTIVE",
  joinedAt: Date,
  removedAt: null,
  removedBy: null
}
```

Design notes: On rejoining, reactivate the existing userId+weddingId membership instead of creating a duplicate. Both admins are represented as two ADMIN membership records.

5.4. managerInvitations

Purpose: Pending and accepted invitations for a user to join a wedding as a manager.

Field	Description
weddingId	Wedding being joined.
emailNormalized	The email address invited.
tokenHash	Hash of a cryptographically random, single-use token.
expiresAt	Seven days after issue.
status	PENDING, ACCEPTED or EXPIRED.
acceptedBy / acceptedAt	Acceptance details.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  emailNormalized: "manager@example.com",
  tokenHash: "...",
  invitedBy: ObjectId,
  expiresAt: Date,
  status: "PENDING",
  acceptedBy: null,
  acceptedAt: null,
  createdAt: Date
}
```

Design notes: Check expiry and status during acceptance; enforce email match and one-time use. Keep invitation history rather than relying on TTL deletion.

5.5. weddings

Purpose: Top-level workspace record and lifecycle state.

Field	Description
brideName / groomName	Names shown on the workspace and public landing page.
weddingDate	Optional until supplied.
venue	Small embedded venue object: name, address and mapUrl.
timezone	Default Asia/Kolkata.
lifecycle	ACTIVE, COMPLETED or ARCHIVED.
publicPageTokenHash	Hash for an unlisted public landing-page link.
lifecycleRequest	Embedded current request and approvals for a two-admin transition.
createdBy	Creator user reference.

```
{
  _id: ObjectId,
  brideName: "Bride Name",
  groomName: "Groom Name",
  weddingDate: null,
  venue: { name: null, address: null, mapUrl: null },
  timezone: "Asia/Kolkata",
  lifecycle: "ACTIVE",
  publicPageTokenHash: "...",
  lifecycleRequest: {
    action: null,
    initiatedBy: null,
    initiatedAt: null,
    approvedBy: []
  },
  createdBy: ObjectId,
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Keep only the current pending lifecycle request embedded. Require two distinct active Admin members to approve completion, reopening, archiving or restoring.

5.6. categories

Purpose: Wedding-scoped custom categories for guest families, expenses and vendors.

Field	Description
type	GUEST, EXPENSE or VENDOR.
name / normalizedName	Display label and normalized value used for uniqueness.
status	ACTIVE or ARCHIVED.
createdBy	User who created the category.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  type: "GUEST",
  name: "Bride's Family",
  normalizedName: "bride's family",
  status: "ACTIVE",
  createdBy: ObjectId,
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Prefer archiving rather than deleting categories in use. Use a partial unique index for active names, or restore/reuse an archived category with the same name.

5.7. events

Purpose: Scheduled wedding events, including multi-day events and separate guest/private details.

Field	Description
guestInfo	Embedded name/description, startAt/endAt, venue and dress code returned to guests.

Field	Description
internalInfo	Private notes and assigned manager IDs; never include in guest API responses.
status	SCHEDULED or CANCELLED; cancelled events remain visible.
guestInfoVersion	Increment when guest-facing information changes.
weddingId	Tenant boundary.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  name: "Mehendi",
  guestInfo: {
    description: "Mehendi ceremony",
    startAt: Date,
    endAt: Date,
    venue: { name: "Venue Name", address: "Venue Address", mapUrl: "..." },
    dressCode: "Traditional"
  },
  internalInfo: { notes: "Private planning notes", assignedManagerIds: [ObjectId] },
  status: "SCHEDULED",
  guestInfoVersion: 1,
  createdBy: ObjectId,
  updatedBy: ObjectId,
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Store date/time instants as UTC dates and render them using the wedding timezone. Guest-facing and internal fields should be separated at the API serializer boundary.

5.8. guestFamilies

Purpose: One record per invited family, with one primary family contact.

Field	Description
primaryContact	Embedded name, mandatory email and optional phone.
categoryId	Optional reference to a GUEST category.
invitationTokenHash	Hash of permanent unpredictable family invitation token.
status	ACTIVE or DELETED.
deletedAt / deletedBy	Deletion metadata; deleting invalidates the guest link.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  familyName: "Sharma Family",
  primaryContact: {
    name: "Rahul Sharma",
    email: "rahul@example.com",
    emailNormalized: "rahul@example.com",
    phone: "+91..."
  },
  categoryId: ObjectId,
  invitationTokenHash: "...",
  status: "ACTIVE",
  createdBy: ObjectId,
  createdAt: Date,
  updatedAt: Date,
  deletedAt: null,
  deletedBy: null
}
```

Design notes: On deletion, clear/unset invitationTokenHash immediately. Validate email and detect likely duplicates during import, but do not auto-merge families.

5.9. guestEventInvitations

Purpose: Join collection connecting a family to each event it is invited to.

Field	Description
guestFamilyId	Family receiving the invitation.
eventId	Event included in that family invitation.
status	INVITED or REVOKED.
invitedBy / invitedAt	Invitation audit fields.
weddingId	Tenant boundary for both references.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  guestFamilyId: ObjectId,
  eventId: ObjectId,
  status: "INVITED",
  invitedBy: ObjectId,
  invitedAt: Date
}
```

Design notes: Use a unique compound index on weddingId + guestFamilyId + eventId. Validate that the family and event both belong to the same wedding.

5.10. rsvps

Purpose: One updatable RSVP response per invited family per event.

Field	Description
guestFamilyId + eventId	Family/event pair being answered.
response	YES, NO or MAYBE.

Field	Description
respondedAt	Most recent response time.
weddingId	Tenant boundary.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  guestFamilyId: ObjectId,
  eventId: ObjectId,
  response: "YES",
  respondedAt: Date,
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Unique compound index on weddingId + guestFamilyId + eventId. Permit RSVP only when the family is active and has an active invitation for the event.

5.11. expenses

Purpose: Agreed expense record; original amount is immutable after creation.

Field	Description
amountAgreedPaise	Integer amount in paise; ₹80,000 is 8,000,000 paise.
categoryId	Optional EXPENSE category.
eventId / vendorId	Optional references, both wedding-scoped.
attachments	Small embedded receipt metadata when needed.
status	ACTIVE or DELETED.
deletedAt / deletedBy	Recycle-bin metadata.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  title: "Wedding Photography",
  amountAgreedPaise: 8000000,
  currency: "INR",
  categoryId: ObjectId,
  eventId: ObjectId,
  vendorId: ObjectId,
  notes: "Full-day photography",
  attachments: [{ publicId: "...", secureUrl: "...", resourceType: "raw" }],
  status: "ACTIVE",
  createdBy: ObjectId,
  updatedBy: ObjectId,
  deletedBy: null,
  deletedAt: null,
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Managers may add/edit but cannot delete. Admin deletion is soft deletion; admin restoration returns the same record. New costs are new expense records, not amendments to the agreed amount.

5.12. payments

Purpose: Separate instalment/payment records against an expense.

Field	Description
expenseId	Parent expense reference.
amountPaise	Positive integer amount in paise.
paidAt	Payment date.
paymentMethod	e.g. CASH, UPI, BANK_TRANSFER, CARD or OTHER.
receipt	Optional embedded receipt metadata.
recordedBy	User who recorded the payment.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  expenseId: ObjectId,
  amountPaise: 2000000,
  paidAt: Date,
  paymentMethod: "UPI",
  receipt: { publicId: "...", secureUrl: "..." },
  recordedBy: ObjectId,
  status: "ACTIVE",
  createdAt: Date
}
```

Design notes: Keep payment records identifiable. Recommended correction path: admin-controlled void with reason, followed by a replacement record. Paid total should include ACTIVE payments only.

5.13. vendors

Purpose: Wedding-specific saved vendor details and workflow status.

Field	Description
businessName / contact details	Manual vendor listing and contact information.
categoryId	Optional VENDOR category.
quotedAmountPaise	Optional quote in integer paise.
status	SHORTLISTED, CONTACTED, FINALISED or COMPLETED.
recordStatus	ACTIVE or REMOVED; preserves history.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  businessName: "Royal Photography",
  contactPerson: "Amit",
  phone: "+91...",
  email: "amit@example.com",
  address: "Kanpur, Uttar Pradesh",
  categoryId: ObjectId,
  quotedAmountPaise: 9000000,
  notes: "Wedding photography package",
  status: "FINALISED",
  recordStatus: "ACTIVE",
  createdBy: ObjectId,
  createdAt: Date,
  updatedAt: Date,
  removedAt: null,
  removedBy: null
}
```

Design notes: Final agreed amount, payments and outstanding balances should come from linked expense/payment records rather than duplicated vendor financial fields.

5.14. tasks

Purpose: Tasks tied to an event and optionally assigned to multiple users.

Field	Description
eventId	Required event reference.
assigneeUserIds	Array of assigned user IDs.
status	PENDING, IN_PROGRESS or COMPLETED.
completionSource	Optional marker such as USER or WEDDING_COMPLETED.
dueDate	Optional due date.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  eventId: ObjectId,
  title: "Confirm decoration",
  description: "Finalise stage decoration",
  dueDate: Date,
  assigneeUserIds: [ObjectId],
  status: "PENDING",
  completedAt: null,
  completionSource: null,
  createdBy: ObjectId,
  updatedBy: ObjectId,
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: Validate every assignee is an active member of the same wedding. On manager removal, require admin reassignment of that manager's pending/in-progress tasks. Wedding completion system-completes pending tasks.

5.15. galleries

Purpose: One event gallery with independent view and upload link tokens.

Field	Description
eventId	One gallery per event.
viewTokenHash	Hash of unguessable view token.
uploadTokenHash	Hash of separate unguessable upload token.
weddingId	Tenant boundary.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  eventId: ObjectId,
  viewTokenHash: "...",
  uploadTokenHash: "...",
  createdAt: Date,
  updatedAt: Date
}
```

Design notes: A unique compound index on weddingId + eventId ensures one gallery per event. Guest view and upload tokens are bearer credentials and should be treated as secrets.

5.16. galleryFiles

Purpose: One metadata document per Cloudinary photo or video asset.

Field	Description
galleryId / eventId	References to the parent gallery and event.
mediaType	PHOTO or VIDEO.
publicId / assetId	Cloudinary identifiers.
secureUrl	Delivery URL; protect access at application layer as required.
uploadedByType	ADMIN, MANAGER or GUEST.
status	ACTIVE or DELETED.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  galleryId: ObjectId,
  eventId: ObjectId,
  mediaType: "PHOTO",
  storageProvider: "CLOUDINARY",
  publicId: "...",
  assetId: "...",
  secureUrl: "...",
  format: "jpg",
  bytes: 2500000,
  width: 1920,
  height: 1080,
  durationSeconds: null,
  uploadedByType: "GUEST",
  status: "ACTIVE",
  uploadedAt: Date,
  deletedAt: null,
  deletedBy: null
}
```

Design notes: The backend should authorize the upload before issuing a signed Cloudinary upload payload, then verify the resulting asset and persist its metadata. Archive state blocks new uploads.

5.17. emailOutbox

Purpose: Durable email delivery records used with the background worker.

Field	Description
recipientEmail / type	Recipient and notification type.
idempotencyKey	Unique key preventing duplicate logical sends.
payload	Template data needed to render the email.
status	PENDING, PROCESSING, SENT or FAILED.
attempts / nextAttemptAt	Retry control fields.

```
{
  _id: ObjectId,
  weddingId: ObjectId,
  guestFamilyId: ObjectId,
  eventId: ObjectId,
  recipientEmail: "rahul@example.com",
  type: "EVENT_UPDATED",
  idempotencyKey: "...",
  payload: {},
  status: "PENDING",
  attempts: 0,
  nextAttemptAt: Date,
  sentAt: null,
  createdAt: Date
}
```

Design notes: Create outbox records in the same logical operation as the event update where possible. A worker claims and sends pending records, updates status, and retries failures according to policy.

6. Index plan

Create indexes explicitly in Mongoose. Unique indexes are correctness constraints, not just performance optimizations. Confirm index builds against the target Atlas deployment before production use.

Collection	Index	Purpose
users	UNIQUE { emailNormalized: 1 }	One account per normalized email.
users	UNIQUE SPARSE { googleId: 1 }	One account per Google identity.
authSessions	TTL { expiresAt: 1 }	Clean up expired session records.
weddings	UNIQUE SPARSE { publicPageTokenHash: 1 }	Unique public landing-page token hash.
weddingMembers	UNIQUE { userId: 1, weddingId: 1 }	One membership per user per wedding.
weddingMembers	{ weddingId: 1, status: 1, role: 1 }	Find active members by role.
managerInvitations	UNIQUE { tokenHash: 1 }	Unique invitation token hash.
categories	UNIQUE ACTIVE { weddingId: 1, type: 1, normalizedName: 1 }	No duplicate active category name per type.
events	{ weddingId: 1, status: 1, guestInfo.startAt: 1 }	Upcoming/scheduled events.
guestFamilies	UNIQUE SPARSE { invitationTokenHash: 1 }	Resolve a guest link to a family.
guestFamilies	{ weddingId: 1, categoryId: 1, status: 1 }	Filter guest families.
guestEventInvitations	UNIQUE { weddingId: 1, guestFamilyId: 1, eventId: 1 }	Prevent duplicate event invitations.
guestEventInvitations	{ weddingId: 1, eventId: 1, status: 1 }	Find invitees for an event.
rsvps	UNIQUE { weddingId: 1, guestFamilyId: 1, eventId: 1 }	One RSVP per family/event.
rsvps	{ weddingId: 1, eventId: 1, response: 1 }	RSVP totals per event.
expenses	{ weddingId: 1, status: 1, categoryId: 1 }	Category summaries.
expenses	{ weddingId: 1, status: 1, eventId: 1 }	Event-wise expenditure.
payments	{ weddingId: 1, expenseId: 1, paidAt: 1 }	Expense payment history.
vendors	{ weddingId: 1, categoryId: 1, status: 1 }	Vendor filters.
tasks	{ weddingId: 1, eventId: 1, status: 1, dueDate: 1 }	Event task lists.
tasks	{ weddingId: 1, assigneeUserIds: 1, status: 1 }	Assignee task lists.
galleries	UNIQUE { weddingId: 1, eventId: 1 }	One gallery per event.
galleryFiles	{ weddingId: 1, galleryId: 1, status: 1, uploadedAt: 1 }	Paginated gallery loading.
emailOutbox	UNIQUE { idempotencyKey: 1 }	Prevent duplicate email jobs.
emailOutbox	{ status: 1, nextAttemptAt: 1 }	Find emails ready for processing.

7. Data integrity and security rules

7.1 Wedding isolation

- Every wedding-specific document must include weddingId.
- Every read, update and delete query must be scoped by both the record identifier and the active weddingId.
- Validate that referenced events, families, categories, vendors, expenses and galleries belong to the same wedding.

```
// Example: tenant-scoped lookup
await Event.findOne({
  _id: eventId,
  weddingId: activeWeddingId
});
```

7.2 Authentication and membership revocation

- Do not rely only on a role embedded in an old JWT. Check active membership in weddingMembers for each protected wedding request, or use a carefully designed short cache with immediate invalidation.
- When a manager is removed, mark membership REMOVED and revoke relevant server-side sessions or membership-scoped access. Active access must stop immediately.
- On manager removal, the admin must reassign pending and in-progress tasks before the workflow is considered resolved.

7.3 Guest and gallery links

- Generate long, unpredictable tokens using a cryptographically secure random generator; store only hashes.
- Check family status and event invitation before returning guest-specific information.
- Keep gallery view and upload tokens separate. A view token must never authorize uploads.
- Validate lifecycle and permissions before issuing a signed Cloudinary upload authorization.

7.4 Financial integrity

- Store INR as integer paise; validate positive integer amounts and avoid JavaScript floating-point arithmetic for money.
- Treat amountAgreedPaise as immutable after expense creation.
- Record instalments separately. Calculate paid and outstanding from non-void payments.
- Managers cannot delete expenses. Admin deletion is soft deletion with deletedBy and deletedAt; restoration restores the same record.

7.5 Lifecycle controls

- ACTIVE allows normal workspace operations and uploads.
- COMPLETED is operationally read-only; gallery uploads remain enabled for the permitted roles and guests.
- ARCHIVED is read-only and blocks all new gallery uploads.
- Complete/reopen/archive/restore transitions require approval from both active admins. Store the current request and distinct approvers.

7.6 API privacy

- Guest event endpoints must return only guestInfo; never serialize internalInfo.
- Never return passwordHash, refreshTokenHash, invitation token hashes or gallery token hashes.
- Use explicit response DTOs/serializers instead of returning raw Mongoose documents.

8. Main workflows and database operations

Workflow	Main operations
Create wedding	Insert wedding and initial two Admin memberships in a transaction where supported.
Invite manager	Insert manager invitation with hash, email, expiry and inviter.
Accept manager invitation	Validate token, expiry and email; create/reactivate membership; mark invitation accepted atomically.
Create event	Insert event with separate guestInfo and internalInfo fields.
Invite family to event	Insert one guestEventInvitations record per family/event pair.
Submit RSVP	Upsert the unique family/event RSVP after checking active invitation.
Record expense	Insert expense with immutable agreed paise amount.
Record payment	Insert a payment record linked to an expense.
Remove manager	Revoke membership/session access and require admin task reassignment.
Delete guest family	Mark deleted and unset invitation token hash; guest link stops working.
Update event guest details	Update event, increment guestInfoVersion and enqueue emailOutbox records for invited families.
Complete wedding	After both admin approvals, transition lifecycle and system-complete pending tasks.
Archive wedding	After both admin approvals, transition lifecycle and disable gallery uploads.

9. Dashboard calculations

Metric	Calculation
Total expenditure	Sum amountAgreedPaise for ACTIVE expenses.
Total paid	Sum amountPaise for ACTIVE payments attached to ACTIVE expenses.
Outstanding	Total expenditure minus total paid.
Upcoming events	SCHEDULED events with a future guestInfo.startAt.
Pending tasks	Count tasks with status PENDING.
In-progress tasks	Count tasks with status IN_PROGRESS.
Potential guest estimate	Number of invited families multiplied by 3.
RSVP attendance estimate	Number of invited families with YES RSVP multiplied by 3.

Recommendation: calculate dashboard values using MongoDB aggregation initially. Introduce cached summaries only if real usage shows a performance need; do not duplicate totals prematurely.

10. Mongoose implementation conventions

- Use ObjectId references for relationships and define ref names consistently.
- Enable timestamps where createdAt/updatedAt are meaningful.
- Define enums for roles, lifecycle, statuses, categories, RSVP responses and payment methods.
- Create unique and compound indexes explicitly in schema definitions.
- Validate cross-collection references and wedding ownership in the service layer.
- Keep database access inside repository modules, consistent with the approved modular-monolith architecture.
- Use MongoDB transactions for multi-document operations where the Atlas deployment supports them.
- Paginate potentially large collections such as guest families, gallery files, expenses and email outbox records.

11. Open implementation decisions

Decision still to finalize	Recommended direction
Second Admin onboarding	Define a secure invite/claim flow that creates the second ADMIN membership; do not treat a manager invitation as an admin invitation.
Payment correction	Use admin-controlled void-and-replacement records with a mandatory reason; avoid silent overwrites.
Exact token/session expiry	Set access-token and refresh-session lifetimes, cookie settings, CSRF protections and rotation policy in the auth implementation.
Concurrent RSVP updates	Use a unique index and atomic upsert; define last-write-wins behavior for simultaneous updates.
Event notification deduplication	Use a stable idempotency key per event/version/recipient/notification type.
Gallery file lifecycle	Define file size/type safeguards and Cloudinary cleanup behavior for deleted metadata/assets.
Backup and recovery	Finalize before production with live wedding data; previously deferred in the system design.

12. Scope and assumptions

- Designed for the WeddingOS MVP and an initial scale target of approximately 10-50 weddings, with up to 500 guest families per wedding.
- Uses MongoDB Atlas and Mongoose, as selected in the System Design Document.
- Uses Cloudinary for media storage; MongoDB stores metadata and provider identifiers, not binary file contents.
- Does not introduce AI, payment processing, UPI/bank integration, guest accounts, vendor accounts, activity feeds, budgets or advanced analytics.
- The database design is a logical blueprint. Field types, validation, serializers, transactions and operational safeguards must be implemented and tested in code.

Appendix A. Naming and status conventions

Domain	Recommended values
Member role	ADMIN, MANAGER
Membership status	ACTIVE, REMOVED

Domain	Recommended values
Wedding lifecycle	ACTIVE, COMPLETED, ARCHIVED
Category type	GUEST, EXPENSE, VENDOR
Event status	SCHEDULED, CANCELLED
Guest invitation status	INVITED, REVOKED
RSVP response	YES, NO, MAYBE
Expense status	ACTIVE, DELETED
Payment status	ACTIVE, VOID
Vendor workflow status	SHORTLISTED, CONTACTED, FINALISED, COMPLETED
Vendor record status	ACTIVE, REMOVED
Task status	PENDING, IN_PROGRESS, COMPLETED
Gallery file status	ACTIVE, DELETED
Email status	PENDING, PROCESSING, SENT, FAILED

Appendix B. Design acceptance checklist

- Every wedding-specific collection contains weddingId.
- Unique indexes prevent duplicate memberships, invitations, RSVPs and galleries.
- Guest and manager link tokens are stored only as hashes.
- Admin/Manager authorization is checked against current membership.
- Guest APIs cannot expose event internalInfo.
- Expense amounts are integer paise and agreed amounts cannot be silently changed.
- Payments are separately identifiable and dashboard totals are derived consistently.
- Completed and Archived lifecycle rules are enforced at API and upload authorization layers.
- Large lists are paginated and common access patterns have supporting indexes.
- Transactions and failure handling are tested for multi-document workflows.